

WHEN THE OUTPUT FEATURE IS A NUMBER

Decision Tree Regression

The same tree, the same binary splits. Entropy and information gain cannot score them any more, so we score them with variance instead.

Entropy Counts Categories. A Salary Has None.

CLASSIFIER · WHAT WE USED

Entropy / Gini measured how mixed a node was.

Information gain picked which feature to split on.

$$H = -\sum p_i \log_2 p_i \quad G = 1 - \sum p_i^2$$

REGRESSOR · WHY THEY FAIL

Both formulas need p_i , the proportion of each class. A continuous target has no classes.

Scores of 52, 56, 58, 62 ... means every row is its own category, so entropy is maximal no matter how you split.

A measure that gives the same answer for every candidate split cannot choose between them.

Unchanged: binary splits (the CART algorithm), greedy top-down growth, one threshold per node. Changed: the impurity measure, and what a leaf predicts.

Ten Students, One Continuous Target

TUTORING HRS	MODE	EXAM SCORE
1.0	Online	52
1.5	In-person	56
2.0	Online	58
2.5	Online	62
3.0	In-person	67
3.5	Online	71
4.0	In-person	75
4.5	In-person	79
5.0	In-person	83
6.0	Online	87

Inputs: tutoring hours (numerical) and mode (categorical). **Output:** exam score out of 100.

The target is a number on a continuous scale, so this is a regression problem and we need a decision tree regressor.

Mean of all ten scores $\hat{y} = 69$ Every calculation on the next slides starts from a mean like this one.

■ THE REPLACEMENT

Variance Is the Impurity Measure for Numbers

$$\text{Var} = \frac{1}{n} \sum (y - \hat{y})^2$$

y each target value in the node

$\hat{y}$ the mean of that node

n how many rows the node holds

This is the mean squared error of the node, since the node predicts its own mean.

READING THE NUMBER

Low variance · values sit close together. A pure node.

High variance · values scattered. A mixed node.

Same job entropy did, just on a number line instead of a set of labels. A node holding 70, 71, 72 is pure. A node holding 52, 70, 87 is not.

Variance of the Root = 129.2

root = [52, 56, 58, 62, 67, 71, 75, 79, 83, 87] $\hat{y} = 69$

$\text{Var}(\text{root}) = 1/10 [(52-69)^2 + (56-69)^2 + (58-69)^2 + (62-69)^2 + (67-69)^2 + (71-69)^2 + (75-69)^2 + (79-69)^2 + (83-69)^2 + (87-69)^2]$

$= 1/10 [289 + 169 + 121 + 49 + 4 + 4 + 36 + 100 + 196 + 324] = 1292 / 10$

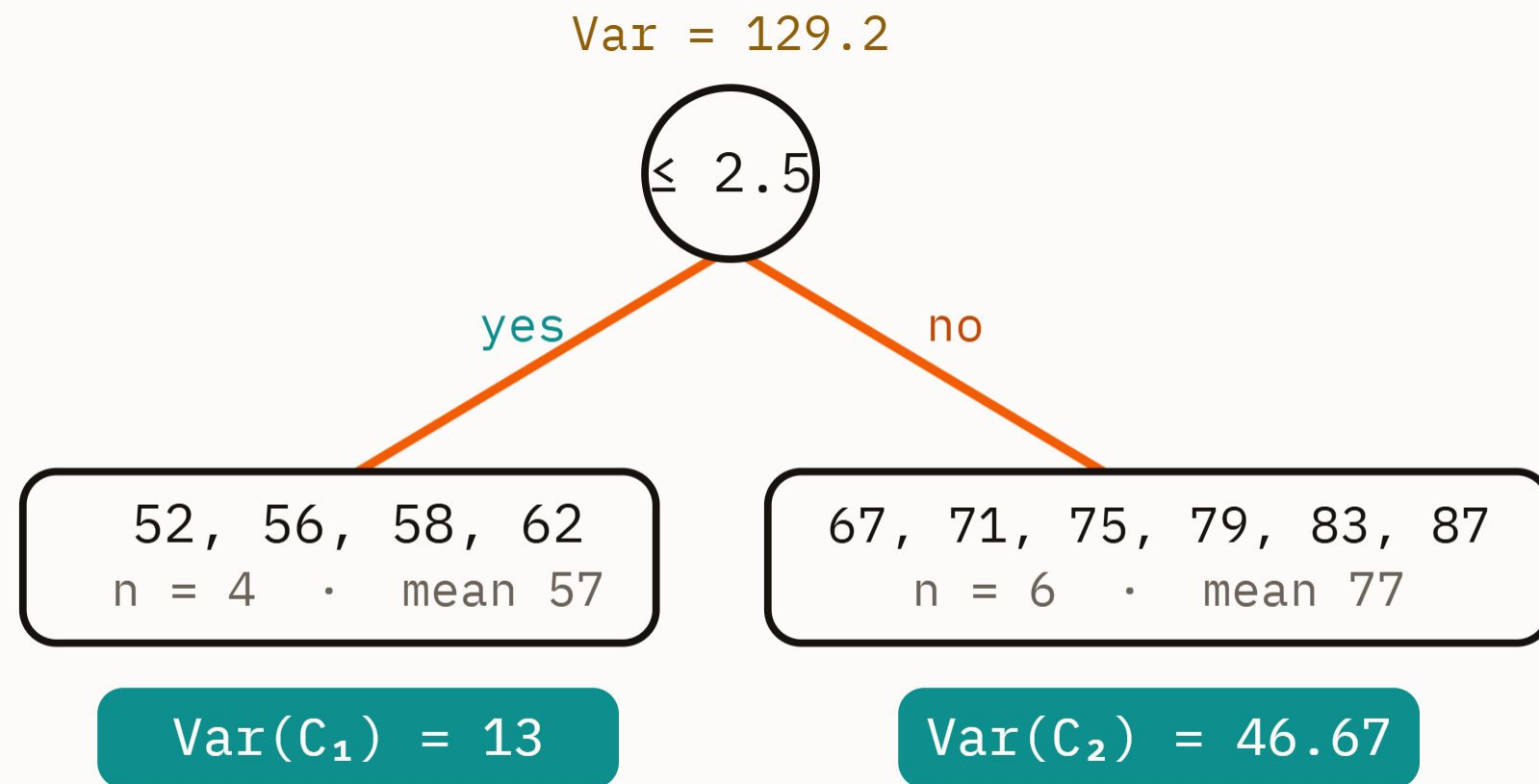
$\text{Var}(\text{root}) = 129.2$

Squaring does two things: it removes the sign, and it punishes far-away values much harder than near ones.

52 and 87 contribute 289 and 324. The two rows nearest the mean contribute 4 each.

129.2 is the number to beat. A split is worth making only if the nodes below it are, on average, tighter than this.

Try hours ≤ 2.5



$$\text{Var}(C_1) = 1/4 [(52-57)^2 + (56-57)^2 + (58-57)^2 + (62-57)^2] = 1/4 [25 + 1 + 1 + 25] = 52/4 = \mathbf{13}$$

$$\text{Var}(C_2) = 1/6 [(67-77)^2 + (71-77)^2 + (75-77)^2 + (79-77)^2 + (83-77)^2 + (87-77)^2] = 1/6 [100+36+4+4+36+100] = 280/6 = \mathbf{46.67}$$

Each child is computed against **its own mean**, not the root's. 57 on the left, 77 on the right.

Variance Reduction: Information Gain for Numbers

$$VR = \text{Var}(\text{parent}) - \sum w_i \cdot \text{Var}(\text{child}_i)$$

w_i is the share of the parent's rows that landed in that child:
4/10 left, 6/10 right.

$$VR = 129.2 - [(4/10 \times 13) + (6/10 \times 46.67)] = 129.2 - [5.2 + 28.0] = 129.2 - 33.2$$

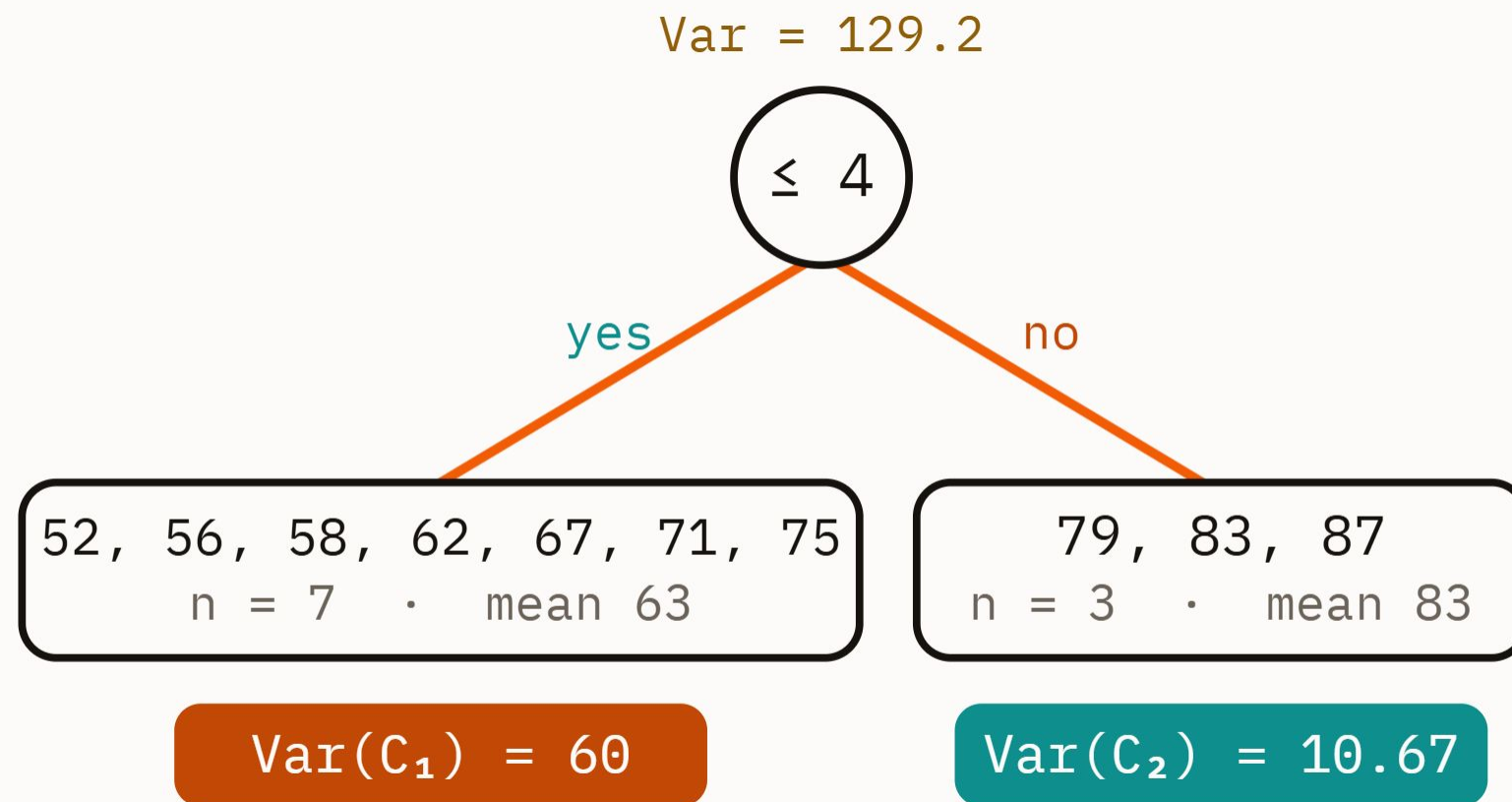
$$VR(\leq 2.5) = 96.0$$

Why weight at all? A child holding one row has variance 0 automatically. Without weights, splitting off single rows would always win.

Structurally identical to information gain. Only the impurity function inside it changed, from entropy to variance.

Impurity before, minus impurity after. **More reduction means a better split**, and the node takes the best one it can find.

Try hours ≤ 4



$$\text{Var}(C_1) = 1/7 [121 + 49 + 25 + 1 + 16 + 64 + 144] = 420/7 = \mathbf{60} \text{ deviations from mean 63}$$

$$\text{Var}(C_2) = 1/3 [(79-83)^2 + 0 + (87-83)^2] = 32/3 = \mathbf{10.67}$$

$$\text{VR} = 129.2 - [(7/10 \times 60) + (3/10 \times 10.67)] = 129.2 - [42.0 + 3.2] = \mathbf{84.0}$$

The right child is very tight, but it holds only 3 of 10 rows, so the weights give it little credit.

■ STEP 5 · THE DECISION

The Higher Variance Reduction Wins

SELECTED

`hours ≤ 2.5`

VR = 96.0

Both children tighter than the parent, and the bigger child carries most of the improvement.

REJECTED

`hours ≤ 4`

VR = 84.0

One very tight child of 3 rows, one still-scattered child of 7 rows that dominates the weighting.

IN PLAIN TERMS

Both splits try to sort one jumbled shelf into two tidier ones.

≤ 2.5 leaves both shelves tidy. ≤ 4 leaves one perfect shelf and one still jumbled, and the jumbled one holds most of the books.

In practice the algorithm does not try two thresholds. It tries every candidate value in the feature, computes this same arithmetic for each, and keeps the highest.

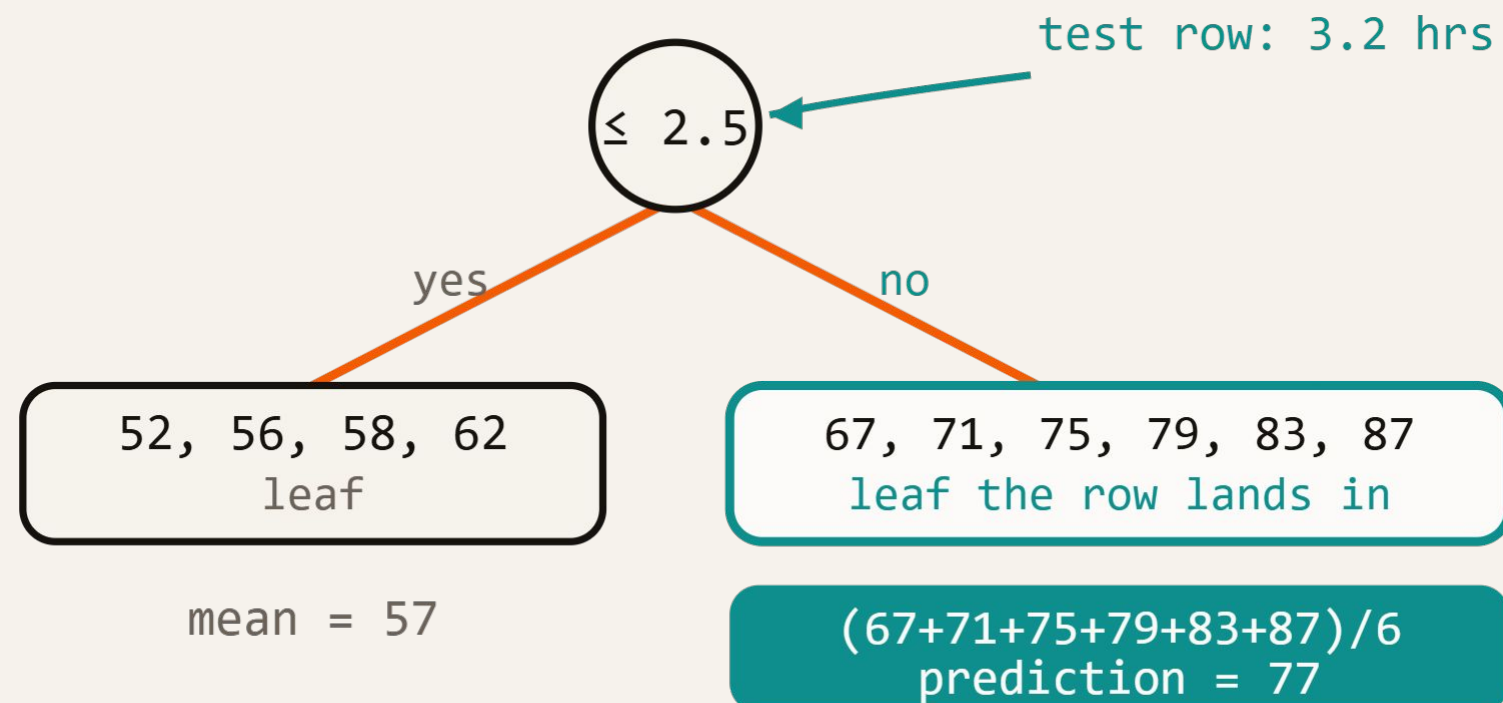
What if We Split on Mode Instead?

$$\begin{aligned} \text{VR} &= 129.2 - [(5/10 \times 148.4) + (5/10 \times 92.0)] \\ &= 129.2 - [74.2 + 46.0] = 129.2 - 120.2 = \mathbf{9.0} \end{aligned}$$

One online child even ends up with variance **higher** than the root: it mixes the lowest score, 52, with the highest, 87.

mode: VR = 9.0 vs hours ≤ 2.5 : VR = 96.0 The root splits on tutoring hours.

A Leaf Returns the Mean, Not a Vote



Classifier leaf: the majority class. **Regressor leaf:** the mean of the training targets that landed there.

3.2 hours is not ≤ 2.5 , so the row goes right and receives 77. A student with 1.8 hours goes left and receives 57.

This tree can only ever answer **57 or 77**. A regression tree predicts in steps, and it has exactly as many possible outputs as it has leaves.

Each Child Repeats the Same Procedure

Treat the child as the new parent. Compute its variance, try every threshold in every feature, take the largest variance reduction. Repeat.

Every extra level doubles the number of distinct values the tree can output, so the step function gets finer.

Unpruned, it ends with **one row per leaf**: variance 0 everywhere, every training score predicted exactly, and nothing learned.

STOPPING RULES CARRY OVER

`max_depth` cap the number of levels

`min_samples_split` do not split a small node

`min_samples_leaf` no leaf below this size

`min_impurity_decrease` refuse splits whose variance reduction is below a floor

Pre-pruning and post-pruning work exactly as in the classifier. Only the impurity being measured is different.

SIDE BY SIDE

Four Rows Change. The Rest Is the Same Tree.

	CLASSIFIER	REGRESSOR
Output feature	Fixed set of categories	Continuous number
Impurity	Entropy or Gini impurity	Variance (mean squared error)
Split score	Information gain	Variance reduction
Leaf output	Majority class	Mean of the rows
Splits, search, pruning	Binary CART splits, greedy top-down search, pre- and post-pruning	Identical in both

■ TAKEAWAYS

The Same Tree, a Different Ruler

01

Variance replaces entropy: the mean squared error of a node against its own mean.

02

Variance reduction replaces information gain, weighted by each child's share of rows. Highest wins.

03

A leaf predicts the mean of its rows, so the output is a step function with one value per leaf.

04

Splits, greedy search and pruning are unchanged from the classifier.

Can the same feature be split twice, at two thresholds? Yes, and for a numeric feature it usually is. A deeper node re-tests the same column on the rows it now holds, which is how the steps get finer.